

Retrieval-based Voice Conversion

The architecture and training of a VITS-derived, GAN-trained retimbre model

A conceptual walkthrough of RVC v2 — grounded in the `rvc` pure-Rust reimplementation

Abstract. — RVC (Retrieval-based Voice Conversion) takes a **source** recording and re-renders it in a **target** voice, keeping **what was said and how it was pitched** while replacing **who is saying it**. This document explains the ideas that make that possible: how the model separates linguistic content and pitch from speaker timbre, how it is structured as a conditional variational autoencoder wrapped around a neural vocoder, and how a **generative adversarial network** is used to teach that vocoder to synthesise waveforms indistinguishable from real audio. Throughout, the concepts are anchored to a concrete open implementation so the abstract picture stays checkable against running code.

Contents

1	What voice conversion actually is	1
2	Analysis: turning audio into content and pitch	3
2.1	ContentVec — a speaker-blind content representation	3
2.2	RMVPE — a robust pitch tracker	3
3	The generative core: a conditional VAE around a vocoder	3
3.1	The two encoders and the latent space	4
3.2	The normalizing flow — bridging prior and posterior	4
4	The vocoder: NSF-HiFiGAN	5
4.1	Why a source-filter model	5
4.2	The deterministic sine source	6
4.3	A note on dimensions (v2, 48 kHz)	6
5	Training I: the reconstruction objectives	7
6	Training II: the generative adversarial network	7
6.1	A tale of two networks	7
6.2	Why many discriminators — multi-period + multi-scale	7
6.3	The four adversarial/perceptual terms, precisely	8
6.4	The alternating step	9
6.5	Reading the loss curves	9
7	Where training goes wrong: data, not just model	9
8	Inference recap: the reverse path	10

1 What voice conversion actually is

A speech signal braids together several independent things at once: the **linguistic content** (the sequence of phonemes — the words), the **prosody** (the pitch contour, or fundamental frequency F_0 , and rhythm), and the **timbre** (the resonances of a particular vocal tract — the thing that makes a

voice recognisable). Text-to-speech generates all three from scratch. Voice conversion is a narrower, in some ways harder, problem: keep content and prosody **exactly**, and swap only the timbre.

The design principle that follows is **disentanglement**. If we can extract a representation of a frame of audio that captures **what is being said** but is **blind to who is saying it**, then we can hand that content-only representation, together with the desired pitch, to a decoder that has learned one specific target voice — and the decoder has no choice but to render the same content in that voice. RVC leans on this hard, and it is why breathy, non-lexical vocalizations (sighs, gasps) survive conversion by construction: they are part of the content and pitch streams, not something the model has to “recognise.”

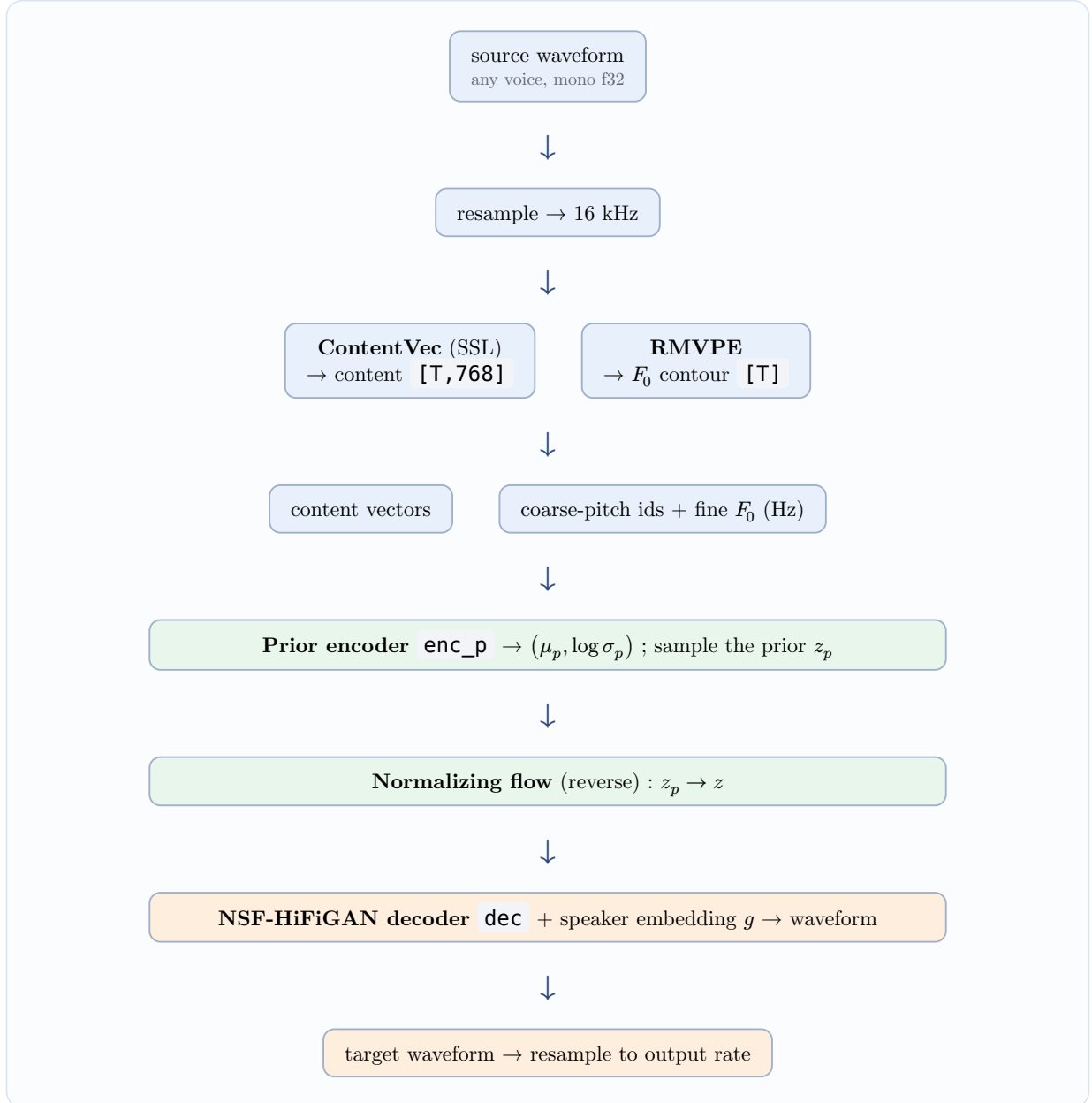


Figure 1: The end-to-end conversion path. Everything above the decoder is speaker-independent analysis; the decoder is the only speaker-specific part.

The rest of this document unfolds that diagram top to bottom, then explains how the speaker-specific decoder is **trained** — which is where the GAN enters.

2 Analysis: turning audio into content and pitch

Two pretrained, frozen models do the analysis. Neither is trained by RVC; they are universal front-ends, run identically at training and inference time (in this implementation they are shared ONNX sessions behind one `FeatureExtractor`).

2.1 ContentVec — a speaker-blind content representation

ContentVec is a **self-supervised** speech encoder (a HuBERT-family transformer) that maps 16 kHz audio to a sequence of 768-dimensional vectors, roughly one per 20 ms frame. Its crucial property for our purposes is that it was trained with a speaker-disentanglement objective: two people saying the same words produce nearly the same ContentVec sequence. That vector is therefore a good proxy for “linguistic content, timbre removed” — exactly the input a voice converter wants.

In code the content stream is extracted at 50 Hz and **upsampled $\times 2$** to the 100 Hz frame grid so it aligns one-to-one with the pitch stream:

```
let up = upsample_rows(&feats.content, 2);    // 50 Hz -> 100 Hz
let n = up.len().min(feats.f0.len());         // align to F0 length
```

2.2 RMVPE — a robust pitch tracker

RMVPE estimates the fundamental frequency F_0 — the pitch — for each frame, and marks frames as **voiced** or **unvoiced** (silence / breath has no pitch). It is prized for staying accurate on noisy, expressive, or quiet material, which matters enormously for ASMR-style corpora full of soft phonation.

Pitch feeds the model in **two** forms, and understanding why is worth a moment:

Coarse pitch (integer ids [1,255]) the continuous F_0 in Hz is warped onto a **mel** scale and quantized into 255 bins. This becomes an **embedding lookup** inside the prior encoder — it is treated like a discrete symbol, letting the network learn a smooth notion of “pitch class.”

Fine F_0 (Hz, continuous) the raw value drives the decoder’s harmonic oscillator directly, so the synthesised waveform sits at **precisely** the right pitch, not the nearest bin.

The mel-warping quantization (from `dsp::f0_to_coarse`) is:

$$\text{mel}(f) = 1127 \cdot \ln\left(1 + \frac{f}{700}\right), \quad \text{id} = \text{round}(\text{clamp}(1..255)).$$

Unvoiced frames ($f = 0$) map to id 1; the map is monotonic, so higher pitch always yields a higher (or equal) bin.

3 The generative core: a conditional VAE around a vocoder

RVC’s network is a descendant of **VITS**, adapted for conversion and named `SynthesizerTrnMs768NSFsid`. It is best understood as a **conditional variational autoencoder** (CVAE) whose decoder happens to be a full neural vocoder. Five submodules cooperate:

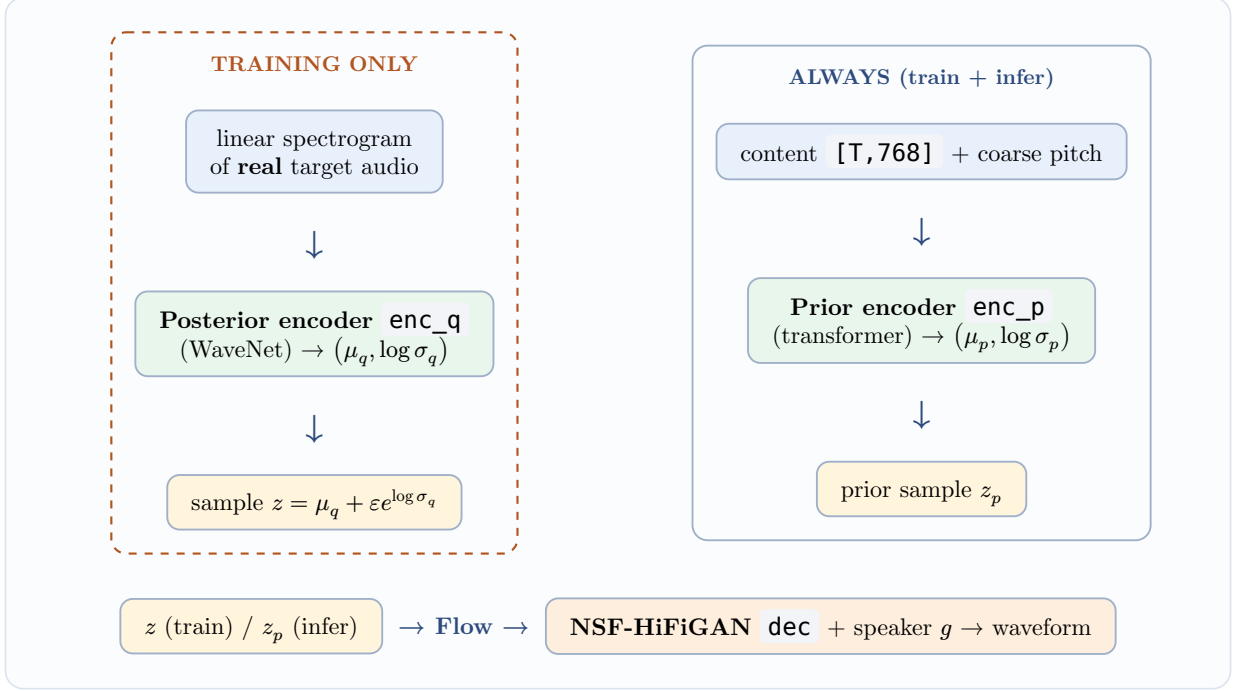


Figure 2: The five modules of the synthesizer and the two data paths. The posterior encoder (dashed) exists **only during training** — it is the “cheat sheet” that teaches the prior what a good latent looks like.

3.1 The two encoders and the latent space

Both encoders emit a **distribution**, not a point: a per-frame mean μ and log-standard-deviation $\log \sigma$ over a 192-dimensional latent. This is the VAE idea — the latent z is stochastic, sampled with the reparameterization trick $z = \mu + \varepsilon \cdot e^{\log \sigma}$, $\varepsilon \sim \mathcal{N}(0, 1)$.

- The **posterior encoder enc_q** sees the **real answer**: the linear spectrogram of the ground-truth target waveform. It is a stack of 16 dilated WaveNet layers and produces the posterior $(\mu_q, \log \sigma_q)$. Because it gets the actual audio, its latent z is easy for the decoder to turn back into that audio. But at inference we do **not** have the target audio — that is the whole point — so **enc_q** cannot be used then.
- The **prior encoder enc_p** sees only **content + pitch** — the information we **will** have at inference. It is a 6-layer relative-position transformer producing the prior $(\mu_p, \log \sigma_p)$.

The trick that ties them together is a **normalizing flow**.

3.2 The normalizing flow — bridging prior and posterior

There is a gap: the posterior latent z (rich, from real audio) and the prior z_p (from content only) live in different-looking distributions. The flow is an **invertible** neural function f_g (conditioned on the speaker g) that warps between them, and invertibility is what lets us run it **both directions**:

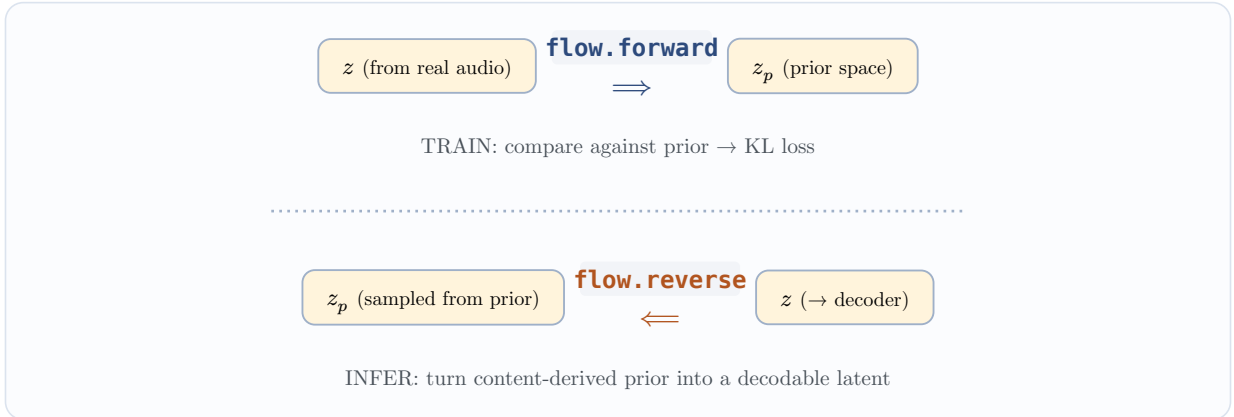


Figure 3: The flow is one function used two ways. Training pushes the posterior latent toward the prior; inference pulls a prior sample into the decoder’s latent space.

RVC’s flow is a `ResidualCouplingBlock` of four **additive coupling layers**. Each layer splits the channels in half, leaves the first half untouched, and adds to the second half a WaveNet-predicted shift computed **from the first half**:

$$x_1 \leftarrow x_1 + m(x_0, g) \quad (\text{forward}), \quad x_1 \leftarrow x_1 - m(x_0, g) \quad (\text{reverse}).$$

Because the shift depends only on the **untouched** half, the layer is trivially invertible — reverse just subtracts the very same predicted mean. A channel **flip** between layers ensures every dimension eventually gets transformed. The layers are `mean_only`, so the log-determinant is zero and there is nothing extra to track:

```
pub fn forward(&self, x, g) { cat(x0, x1 + mean(x0, g)) } // add
pub fn reverse(&self, x, g) { cat(x0, x1 - mean(x0, g)) } // subtract
```

At inference the sampling temperature is deliberately cooled — $z_p = \mu_p + \sigma_p \cdot \varepsilon \cdot 0.666$ — trading a little diversity for cleaner, more stable output.

4 The vocoder: NSF-HiFiGAN

The decoder `dec` turns the 192-channel latent (at 100 Hz) into a waveform at 48 kHz. That is a $480\times$ increase in sample rate — the **hop length**, the product of the upsample factors $12 \cdot 10 \cdot 2 \cdot 2 = 480$. It is a **HiFi-GAN** generator with a **Neural Source-Filter** (NSF) twist.

4.1 Why a source-filter model

Classic HiFi-GAN upsamples the latent through transposed convolutions and lets the network invent all the fine structure. For **singing and expressive speech** that struggles: the network has to hallucinate an exact pitch. NSF fixes this by **handing the network the pitch as an explicit excitation signal**. The `SourceModule` builds a sine wave whose instantaneous frequency tracks the fine F_0 contour — a clean harmonic **source** — and injects it into every upsampling stage. The convolutional stack then acts as a learned **filter** shaping that source into a natural voice. Pitch accuracy comes for free because it is built into the source.

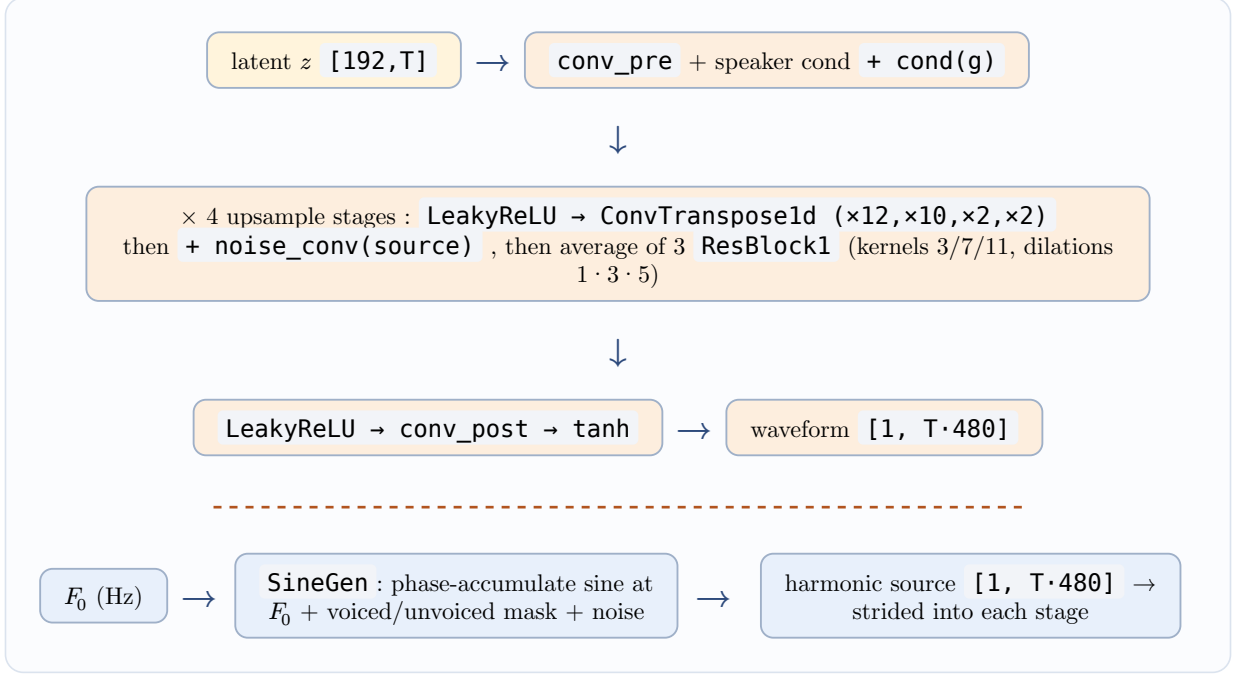


Figure 4: Inside the NSF-HiFiGAN decoder. The sine source (bottom) is generated deterministically from F_0 and mixed in at each upsampling stage; the speaker embedding g conditions the head.

4.2 The deterministic sine source

The sine generator is **not learned** (RVC runs it under `no_grad`; here it is plain arithmetic). For each frame it computes a phase increment f_0/sr , accumulates phase across the whole clip, upsamples the cumulative phase (linear, `align_corners`) and takes its sine — carefully detecting phase wraps so the oscillator stays continuous:

```
phase += rad_up[j] + shift;           // shift = -1 on a detected wrap
sine[j] = (phase * 2.0 * PI).sin() * SINE_AMP;
```

Voiced frames get the sine; unvoiced frames get low-level Gaussian noise (breath). The only learnable part of the source is a single `l_linear` projection feeding a `tanh`. This is precisely why **breathy, unvoiced ASMR content survives**: unvoiced frames are handled explicitly by the noise branch, not discarded.

4.3 A note on dimensions (v2, 48 kHz)

quantity	value	meaning
content dim	768	ContentVec output width, per frame
latent <code>inter_channels</code>	192	shared width of prior / posterior / flow
<code>hidden_channels</code>	192	transformer & WaveNet width
<code>spec_channels</code>	1025	linear-spectrogram bins ($\frac{n_m}{2} + 1$) into <code>enc_q</code>
<code>gin_channels</code>	256	speaker-embedding width g
transformer	6 layers, 2 heads	prior encoder <code>enc_p</code>
flow	4 coupling layers	additive, <code>mean_only</code> , with flips
upsample rates	$12 \cdot 10 \cdot 2 \cdot 2$	product = hop = 480 samples/frame
frame rate	100 Hz	both content and pitch land on this grid

5 Training I: the reconstruction objectives

Training is **fine-tuning**: warm-start every module from the public pretrained base (`f0G48k.pth` / `f0D48k.pth`), then adapt to one target voice on a small corpus. The module layout is deliberately kept weight-compatible with the reference PyTorch `state_dict` so this warm-start is possible — essential when the corpus is only 1 hour long.

Each step draws a batch of short random windows from the corpus, runs the **training forward** (posterior $\rightarrow$ flow $\rightarrow$ decode a random 0.36 s segment), and computes a compound loss. Three of its terms are **reconstruction** terms; the other two (next section) are **adversarial**.

Mel-spectrogram L1 ($\times 45$) the perceptual backbone. Convert both the real segment and the generated one to a 128-band mel spectrogram and take the mean absolute difference. This is what actually makes the output **sound** like the target; its large weight (45) reflects that.

$$L_{\text{mel}} = \| \text{mel}(y) - \text{mel}(\hat{y}) \|_1 .$$

KL divergence ($\times 1$) the CVAE glue. It pushes the flow-transformed posterior z_p to look like a sample from the prior $(\mu_p, \log \sigma_p)$, so that at inference — when only the prior is available — the decoder still receives a latent it knows how to render. RVC uses VITS’s Monte-Carlo form:

$$L_{\text{kl}} = \log \sigma_p - \log \sigma_q - \frac{1}{2} + \frac{1}{2} (z_p - \mu_p)^2 e^{-2 \log \sigma_p} .$$

Feature matching ($\times 2$) defined via the discriminator, covered below.

A subtle but important detail: the posterior’s input spectrogram is `.detach()`-ed, and in feature matching the **real** features are detached — they are fixed targets, and gradients must not flow into them.

6 Training II: the generative adversarial network

Mel-L1 alone produces **oversmoothed, buzzy** audio: averaging many plausible waveforms that share a mel spectrogram gives a muddy one. The fix is a **GAN**. We introduce a second network — the **discriminator** — whose only job is to tell **real** target audio from the **generated** audio. The generator (our whole synthesizer) is then additionally trained to **fool** it. In the limit, “audio the discriminator can’t distinguish from real” is exactly “audio that has the crisp fine structure of real speech.”

6.1 A tale of two networks

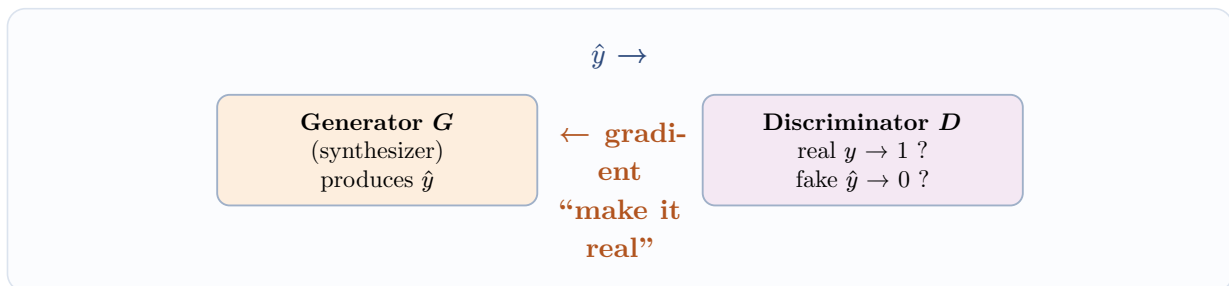


Figure 5: The adversarial game. G wants D to score its output as real; D wants to score real as 1 and fake as 0. They are trained in alternation, each with its own AdamW optimiser.

6.2 Why many discriminators — multi-period + multi-scale

A single discriminator looking at the raw 1-D waveform tends to miss **periodic** artefacts — the buzz and roughness that live in the pitch harmonics. RVC (from HiFi-GAN) therefore uses a **bank** of discriminators, a `MultiPeriodDiscriminatorV2` :

- One **scale** discriminator (**DiscriminatorS**) reads the waveform as a plain 1-D signal — good at broad envelope and noise texture.
- Eight **period** discriminators (**DiscriminatorP**), one for each prime period in $\{2, 3, 5, 7, 11, 17, 23, 37\}$. Each **folds** the 1-D waveform into a 2-D grid with that period as the row stride, then runs 2-D convolutions. Folding on a period exposes structure that repeats at that spacing — precisely where pitch-related artefacts hide. Using **coprime** periods means the discriminators cover a wide range of periodicities without redundantly overlapping.

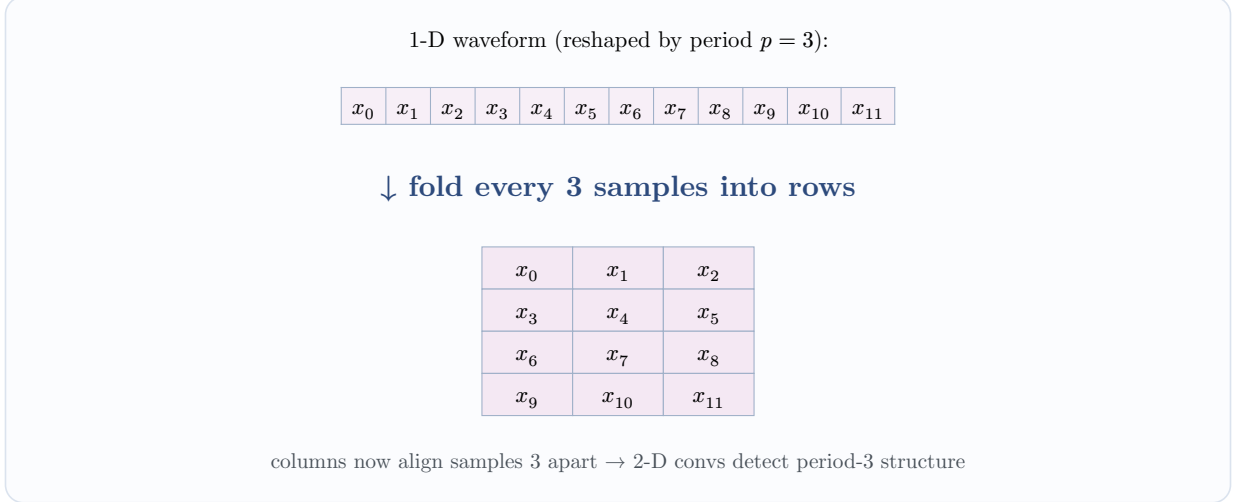


Figure 6: How a period discriminator “sees” the signal: a period-3 fold turns a 1-D strip into a 2-D image whose columns line up periodic samples, then convolves it in 2-D. Eight coprime periods cover many spacings.

Both discriminator families also expose their **intermediate feature maps**, which powers the feature-matching loss.

6.3 The four adversarial/perceptual terms, precisely

RVC uses the **least-squares** GAN (LSGAN) formulation — it is more stable than the log-loss original. Writing D_k for the k -th discriminator’s score:

Discriminator loss (push real→1, fake→0)

$$L(D) = \sum_k \mathbb{E}[(D_k(y) - 1)^2] + \mathbb{E}[D_k(\hat{y})^2].$$

Generator adversarial loss (push fake→1)

$$L_{\text{adv}(G)} = \sum_k \mathbb{E}[(D_k(\hat{y}) - 1)^2].$$

Feature matching ($\times 2$) match G ’s audio to the target **inside** the discriminator, layer by layer — a perceptual L_1 that stabilises training:

$$L_{\text{fm}} = \sum_k \sum_l \left\| D_k^{(l)}(y) - D_k^{(l)}(\hat{y}) \right\|_1.$$

The complete generator objective is the weighted sum

$$L_G = L_{\text{adv}(G)} + 2L_{\text{fm}} + 45L_{\text{mel}} + L_{\text{kl}}.$$

6.4 The alternating step

One training step updates the two networks in turn. The generated audio is **detached** for the discriminator step, so D 's gradient never leaks into G ; then G is updated through the (now fixed-weights) discriminators.

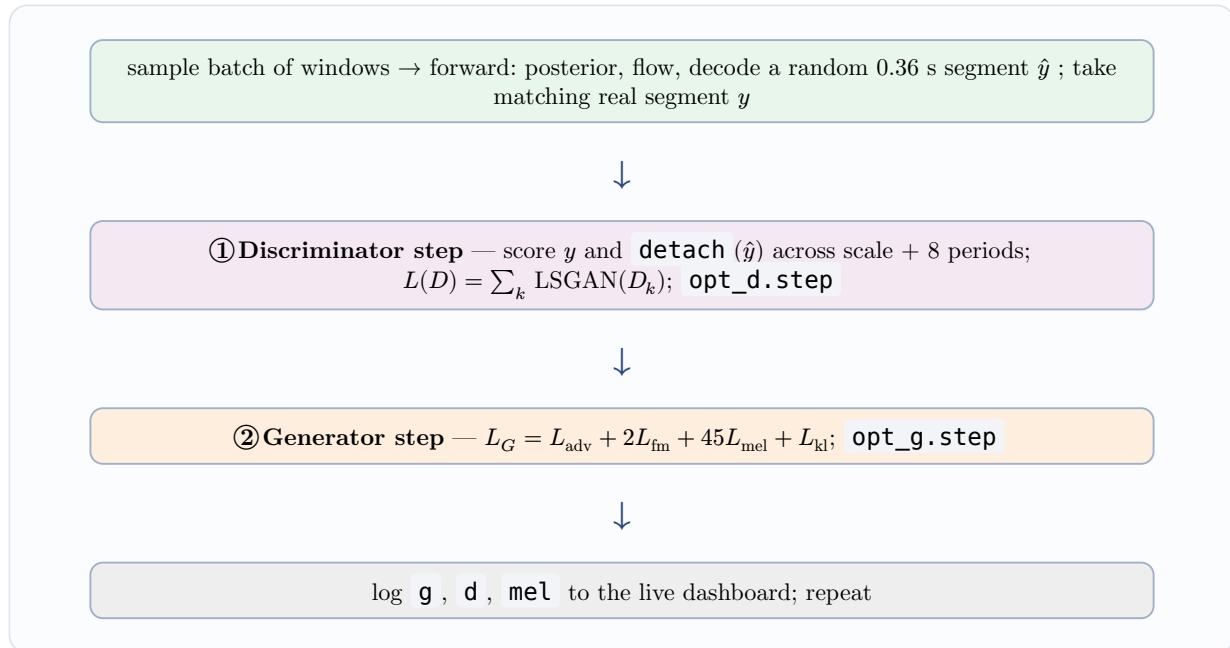


Figure 7: One training iteration. The order matters: D first on detached audio, then G against the updated D .

Both optimisers are AdamW ($\beta_1 \{=\} 0.8$, $\beta_2 \{=\} 0.99$, lr 10^{-4}), matching the reference recipe.

6.5 Reading the loss curves

Because G and D are **adversaries**, their losses do **not** both fall to zero — they seek an equilibrium. Practical intuition:

- **mel_loss** is the one to watch. It is the honest reconstruction signal and **should trend down**. If it shakes around a plateau and never descends, the generator is not learning the voice — most often a **data** problem (see below), not a hyperparameter one.
- **d_loss** near a small positive constant (not collapsing to 0) means the discriminator is appropriately challenged. A **d_loss** crashing to 0 means D has won and G gets no useful gradient.
- **g_loss** is dominated by the $45 \times \text{mel}$ term, so it largely tracks **mel_loss** plus adversarial jitter.

7 Where training goes wrong: data, not just model

The most common failure on real-world corpora is a generator that **collapses to silence** — **mel_loss** stuck and shaking, output pure silence, even though the pretrained base works. The architecture is fine; the **training distribution** is poisoned.

The trainer draws **random short windows uniformly across each file**. If the raw recordings are full of between-sentence dead air, then most randomly sampled 0.36 s segments **are silence**, and the fastest way to minimise mel-L1 on a silence-heavy batch is to **emit silence**. The generator obliges, globally.

The remedy is a **sentence-safe slicer** run as a preprocessing pass (**rvc preprocess**): it uses short-time energy with **hysteresis** to cut only **genuine, sustained** dead air between sentences, while:

- **never splitting a complete sentence** — a gap must be both below the energy floor **and** longer than a minimum duration before it counts as a cut; and
- **preserving soft, breathy content** — energy is used only to **locate** long silent gaps, never to gate quiet-but-present sound, and cut boundaries are edge-padded so onsets and breathy tails are kept.

The result is a corpus of clean per-sentence clips, so uniform window sampling now lands on real phonation. Training itself is unchanged — it simply consumes a healthier distribution.

8 Inference recap: the reverse path

At inference the posterior encoder and the whole discriminator bank are gone. The path collapses to: analyse source → prior encoder → **sample** z_p → **reverse** the flow → decode with the target speaker embedding. Content and pitch came from the **source**; timbre came from the **trained decoder + speaker embedding**. That asymmetry — universal analysis in, speaker-specific synthesis out — is the whole idea of voice conversion, and every architectural choice above exists to keep those two halves cleanly separated.

Concepts anchored to the `rvc` implementation: `burn-rvc` (network), `rvc-core` (feature extraction + DSP + backends), `rvc-train` (adversarial training loop). RVC v2, 48 kHz, weight-compatible with the reference `SynthesizerTrnMs768NSFsid` / `MultiPeriodDiscriminatorV2`.